

DRP

Machine Learning DRP Presentation

TODO¹ TODO² TODO³ TODO⁴ TODO⁵

¹Department of Computer Science

²Department of Statistics

³Department of Mathematics

⁴Department of Electrical Engineering

⁵Department of Cognitive Science

University of Toronto

May 2025

Table of Contents

- 1 Introduction
- 2 Transformers
- 3 Pipeline: Collecting Activations
- 4 Pipeline: Generating Interpretations
- 5 Math Breakdown
- 6 Evaluation

Core Challenge

- They describe LLMs as black boxes with **billions** of uninterpretable features
- Sparse Autoencoders (SAEs):
 - Transform dense activations → sparse latents
 - 1M+ features need interpretation
- Manual analysis → **impossible** at scale

Paper's Goal/Contribution

- Automated process/pipeline for SAE latent interpretation
- 5 new scoring metrics
- Open-source code + 1M+ interpretations

Prerequisite Vocabulary: Transformers for Text Processing

- **Tokens:** basic unit of text processed by transformers. Can be words, subwords, or characters.
- **Embeddings:** map tokens to dense vectors in a high-dimensional space. These vectors capture semantic relationships: similar tokens (e.g., "king" and "queen") have similar vectors.
 - Embeddings turn discrete tokens into continuous vectors, allowing mathematical operations (e.g., king - man + woman = queen).
- **Positional encoding:** encoding the position of each token in a sentence with a sinusoidal function. Added to the embeddings. Allows the model to learn relative positions of tokens.

Attention is All You Need

- "Attention is All You Need" (Vaswani et al, 2017) introduced the Transformer architecture.
- Attention is a mathematical function defined as:
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V, \text{ where}$$
 - Q : Queries matrix. Identifies what a token seeks.
 - K : Keys matrix. Identifies what a token offers.
 - V : Values matrix. Carries the actual information to aggregate based on attention weights
 - d_k : Dimension of keys (scaling prevents gradient saturation)
- Q, K, V : learned projections
- Each query vector q_i computes cosine similarity with all keys k_j , creating attention weights that linearly combine value vectors v_j .

Transformer Architecture

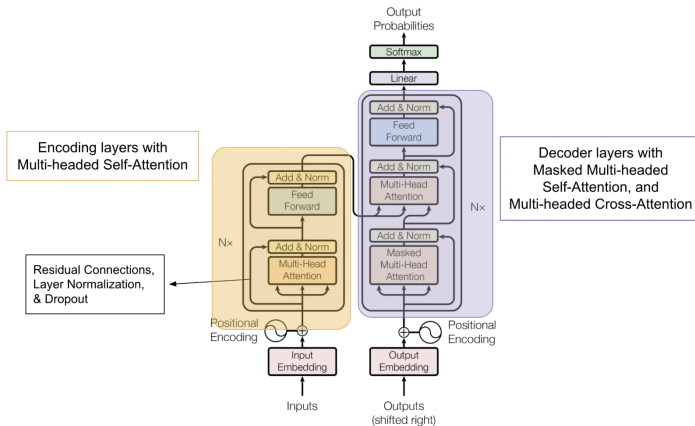


Figure: Transformer architecture.

Attention is All You Need

- Instead of one attention function, transformers use $h=8$ parallel heads:
 $MultiHead(Q, K, V) = MultiHead(XW_i^Q, XW_i^K, XW_i^V) =$
 $Concat(head_1, \dots, head_k)W^0$
- Each head applies attention to linearly projected versions of Q, K, V :
 $head_i = Attention(XW_i^Q, XW_i^K, XW_i^V)$, where W_i^Q, W_i^K, W_i^V are learnable projection matrices.
- Each head learns a unique 'lens' to focus on different relationships (e.g., syntax vs. semantics).

Our project: Interpretability

Mechanistic interpretability: understanding the internal workings of AI models by identifying the specific mechanisms driving their behaviour.
Why study internal model representations?

- Trust and safety: Transformers are "black boxes". Learning how knowledge is represented in them helps build trust.
- Model debugging: Understanding attention patterns reveals how models prioritize features.
- Regulation compliance: Laws (like the EU AI Act) often require explanations for automated decisions. Interpretable transformers enable compliance in sectors like healthcare or finance.

Activation Patterns

- Used RedPajama-v2 as their dataset (10M tokens)
- Batch size = 256 tokens (BOS initialized)
- SAE Configurations:
 - Gemma 2 9B: 16k/131k latents
 - Llama 3.1 8B: 262k latents
- Activation Stats:
 - 131k SAE: 30% latents ;200 activations
 - 15% completely inactive :(
 - 16k SAE: 10% less than 200 activations

Some Important Things to Note

- Larger SAEs → more dataset-specific features
- Training context length (1024) vs eval (256)

Automated Labeling (MAIN method proposed by the paper)

- LLM model for labeling is Llama 3.1 70B Instruct
- Input is 40 activating examples/latent
 - 32 tokens/example
 - Top activating tokens highlighted
- Sampling Strategies:
 - “Top” examples are high precision, low recall
 - “Stratified” are broad but vague
 - “Random” are at least 23% worse accuracy
- Cost = \$200/1M latents

Limitations

- 32-token context → misses long-range patterns
- Layer 1-8: 15% lower accuracy

Core Equations

SAE Objective

$$\mathcal{L} = \underbrace{\|\mathbf{W}_d(\mathbf{W}_e\mathbf{x} + \mathbf{b}_e) + \mathbf{b}_d - \mathbf{x}\|^2}_{\text{Reconstruction}} + \underbrace{\lambda\|\mathbf{h}\|_1}_{\text{Sparsity}}$$

Detection Scoring

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Intervention Scoring

$$\mathcal{S}(I, z) = \mathbb{E}_{x \sim \pi}[D_{KL}(p_C \| p_{\text{intervnd}})]$$

Surprisal Scoring

$$\Delta S = \mathbb{E}[\log p(x|z) - \log p(x|\tilde{z})]$$

Embedding Scoring

$$\text{Sim} = \cos(\mathbf{E}(z), \mathbf{E}(x_{\text{act}}))$$

Conclusion Summary

- 5 New Scoring Metrics:
 - Detection, Fuzzing, Surprisal, Embedding, Intervention
- Important Results:
 - Fuzzing: 0.69 correlation with human eval
 - Intervention: -0.37 correlation with context scores
 - Layer 32: 40% higher scores than Layer 1
- Cost Analysis:
 - Embedding: \$50/100k latents
 - Simulation: 30x more expensive

Conclusion Takeaways

- Automated interpretations help and enable SAE debugging!
- Output features need causal metrics